Software - Architecture, Design, Test, and...

Troels Mortensen

June 4, 2026

Contents

Todo list

Chapter 1

Hexagonal Architecture

Hexagonal Architecture, also known as *Ports and Adapters*, was introduced by Alistair Cockburn to protect application behavior from infrastructure concerns.[1] The key idea is simple: we keep application logic in the center, and let external technologies connect through explicit boundaries. These boundaries are called *ports*, and are generally just interfaces in your code. A modern introduction with practical examples is provided by Huttunen, and has inspired parts of this chapter.[4]

In this chapter, a *port* always means an interface: the contract between the core and the adapters. *Inbound ports* are what driving adapters call to invoke a use case; *outbound ports* are what the core calls when it needs infrastructure capabilities. Adapters on the outside are often called *driving* (or primary) and *driven* (or secondary); literature uses several names, but I try stick to inbound and outbound for ports throughout.

In short, the point of hexagonal architecture is to keep a primary focus on the business logic, and allow external technologies to "plug in" to the core system, through interfaces. It is a "solve the business problem"-first approach, rather than a "technology"-first approach.

Whether the system is activated through a GUI, a Web API, a CLI application, or something else, is considered a technical "detail" of the system.

Whether data is stored in a database, a file, or something else, is considered a technical "detail" of the system.

1.1 Why We Use Hexagonal Architecture

Many systems start in a layered style, but business rules often leak into controllers, persistence code, ORMs, and other technological details. Basically just places where it does not belong. This coupling makes change expensive: replacing persistence technology, adding a new interface, or testing behavior can require touching several technical layers, even if we just want to verify the core business logic is working as expected.

Similarly to a 3-layered architecture, we want to separate the concerns of the application into different parts. We just approach it slightly differently.

Hexagonal architecture addresses this by separating concerns into an *inside* and an *outside*. The inside contains the behavior we care about, while the outside contains delivery and infrastructure details. Comparing to the 3-layered architecture, the presentation and persistence will be considered the "outside" of the application. We start with the core hexagon, see Figure 1.1.

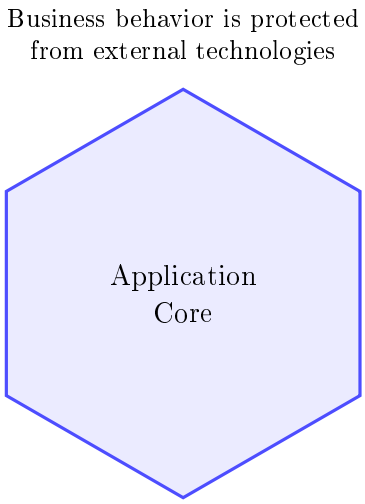


Figure 1.1: The first step is to define a protected application core.

1.1.1 Purpose of the Inner Hexagon

The hexagon is a visual metaphor, not a geometric requirement (which probably would not make sense). Cockburn has said it was simply a shape that was not yet associated with anything in software architecture. It reminds us that the core should be reachable from many directions, or input devices, without being rewritten for each one. Whether requests come from a GUI, a Web API, a CLI application, a batch job, or an automated test, they should trigger the same use-case behavior.

1.2 Step 1: Start with Application Logic

At the beginning, we model only application behavior, for example in `ConfirmKaijuDetectionService` or `TrackKaijuUseCase`. These classes are responsible for business logic, i.e. executing a use case and enforcing its rules. Commonly such a class is suffixed "Service", or "UseCase", or "Handler".

At this stage we do not need to commit to `Spring`, `ASP.NET`, `PostgreSQL`, or a message broker. We intentionally start here because architecture decisions become clearer when behavior is visible first.

For the rest of the chapter, we use one running Java example: `Confirm First Kaiju Detection`. This gives us one consistent slice from driving adapter to core behavior to driven adapters.

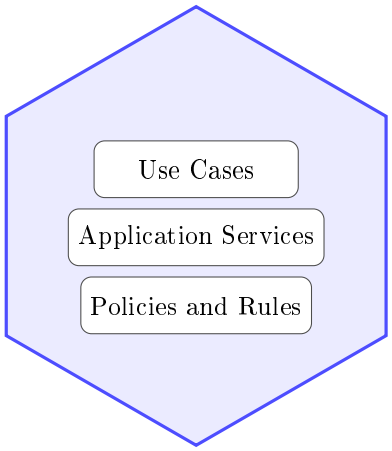


Figure 1.2: Initial core structure focused on use cases and policies.

The content of the application core has several names. Often, classes in here are called "application services" or "use cases". And are therefore suffixes with "Service" or "UseCase". The responsibility of these classes is to execute a use case and enforce its rules. For example, `ConfirmKaijuDetectionService` is a service that confirms a Kaiju detection. The class will then expose a method for this particular use case, and implement the logic for it.

Logic include many things:

- Validating input data,
- Applying business rules,
- Coordinating across outbound ports,
- and persisting the results.

As such, the application core is the heart of the application, and over time, it will grow to contain the majority of the application's logic, placed in these service classes.

1.2.1 What We Delay on Purpose

We deliberately delay technology choices. This gives us three benefits:

- We can validate business behavior early without waiting for full infrastructure setup.
- We avoid coupling use-case logic to framework types and transport protocols.
- and we can define outbound port interfaces from the use case before any adapter exists, based on what the use case requires.

In practice, that means naming what the core *requires* first. For example, a triangulation use case may need to poll sensors and receive readings long before anyone chooses a vendor protocol or wire format. We declare a `SensorDriver` outbound port with that capability, implement the use case against the interface using fakes, and add real adapters when integration details are known. This port-first workflow is the same idea as core-owned contracts in Section 1.4.5, and we apply it again in Section 1.7.

1.2.2 Java Skeleton of the Core

The initial core can be expressed as a use-case-focused Java API and service implementation.

First we need the "entry" interface, i.e. the inbound port that driving adapters will call to invoke the use case. It declares a piece of behaviour the core can execute.

```
1 public interface ConfirmKaijuDetectionPort
2 {
3
4     KaijuId confirmDetection(
5         ConfirmKaijuDetectionCommand command
6     );
7
8 }
```

Code snippet 1.1: `ConfirmKaijuDetectionPort` inbound port

Next, we have the implementation of the port in Code snippet 1.2. It is a service class that implements the interface and contains the use-case logic. Here, we confirm a detection by linking a new `Kaiju` to its origin `Breach` and persisting it through `SaveKaijuPort`.
There is also a second outbound port, `PublishKaijuConfirmedPort`, implemented by a driven adapter that publishes a mission event. It is similar to the Observer Design Pattern. This event can trigger downstream actions such as command alerts or tracking workflows.

IN-REF: add reference to Observer Pattern

```java
public final class ConfirmKaijuDetectionService
    implements ConfirmKaijuDetectionPort
{
    private final LoadBreachPort loadBreachPort;
    private final SaveKaijuPort saveKaijuPort;
    private final PublishKaijuConfirmedPort
        publishKaijuConfirmedPort;

    // Constructor omitted for brevity.

    @Override
    public KaijuId confirmDetection(
        ConfirmKaijuDetectionCommand command
    )
    {
        Optional<Breach> breach = loadBreachPort
            .loadById(command.originBreachId())
            .orThrow();

        Kaiju kaiju = Kaiju.confirmed(
            command.codename(),
            command.classification(),
            breach.id()
        );

        saveKaijuPort.save(kaiju);

        publishKaijuConfirmedPort
            .publishConfirmed(kaiju.id(), breach.id());

        return kaiju.id();
    }
}
```

Code snippet 1.2: `ConfirmKaijuDetectionService` use-case implementation

1.3 Step 2: Add Driving Adapters and Inbound Ports

Once the core behavior exists, we connect ways to invoke it. These are *driving adapters* (also called primary adapters). Their responsibility is to translate external input into calls on inbound ports, which are interfaces representing use cases.
Obviously, multiple driving adapters can be used to invoke the same use case. For example, we could have a REST controller, a CLI handler, and a GUI controller all invoking the same use case. This is why we need the inbound port, to abstract the use case from the driving adapter. In the below diagram I have kept it simple and reduced the number of arrows from driving adapters

to inbound ports.

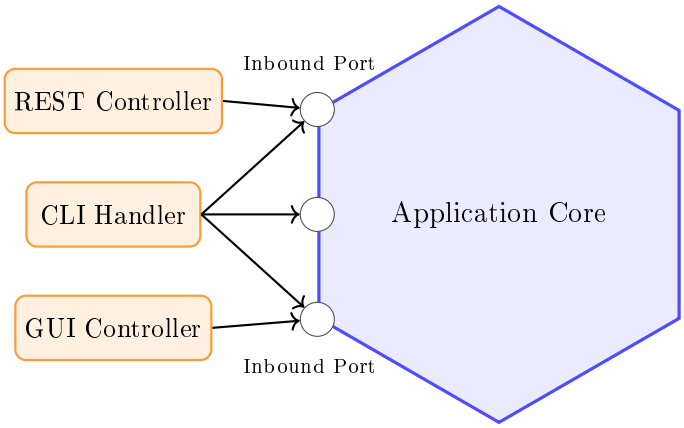


Figure 1.3: Driving adapters call inbound ports to invoke application behavior.

We can simplify the diagram a bit, by just adding a new hexagonal layer to the left side to represent the driving adapters:

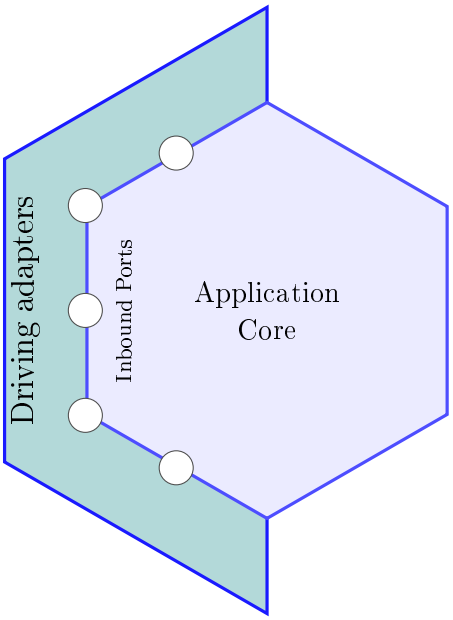


Figure 1.4: Driving adapters to the left of the application core.

1.3.1 Examples of Driving Adapters

Common driving adapters include:

- a REST controller translating HTTP requests into use-case calls,
- a CLI handler translating command arguments into use-case calls,
- a GUI presenter/controller translating user events into use-case calls,
- a message consumer translating broker events into use-case calls,
- and automated tests that call inbound ports directly.

1.3.2 Inbound Port and Driving Adapter in Java

Code snippet 1.3 shows a driving adapter that talks only to the inbound port and maps transport-specific models to use-case models:

```
@RestController
@RequestMapping("/kaiju-detections")
public class KaijuController
{
    // Dependency injected.
    private final ConfirmKaijuDetectionPort
        confirmKaijuDetectionPort;

    @PostMapping
    public ResponseEntity<String> confirm(
        @RequestBody KaijuDetectionRequest request
    )
    {
        ConfirmKaijuDetectionCommand command =
            new ConfirmKaijuDetectionCommand(
                request.codename(),
                request.classification(),
                request.originBreachId()
            );

        KaijuId kaijuId = confirmKaijuDetectionPort
            .confirmDetection(command);

        return ResponseEntity.accepted().body(kaijuId.value());
    }
}
```

Code snippet 1.3: `KaijuController` REST driving adapter

1.4 Step 3: Add Driven Adapters and Outbound Ports

The core eventually needs support from external capabilities such as storage, notifications, or third-party APIs. We model these needs as outbound ports, and implement those ports with *driven adapters* (secondary adapters). This is where dependency inversion becomes concrete: the core depends on interfaces (and *owns* those interfaces), while infrastructure depends on those same interfaces.[3]
Again, we can simplify the diagram a bit, by just adding a new hexagonal layer to the right side to represent the driven adapters:

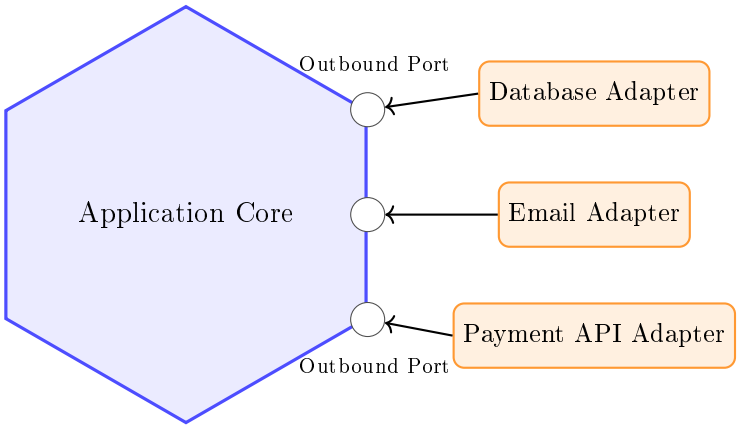


Figure 1.5: Driven adapters implement outbound ports for infrastructure concerns.

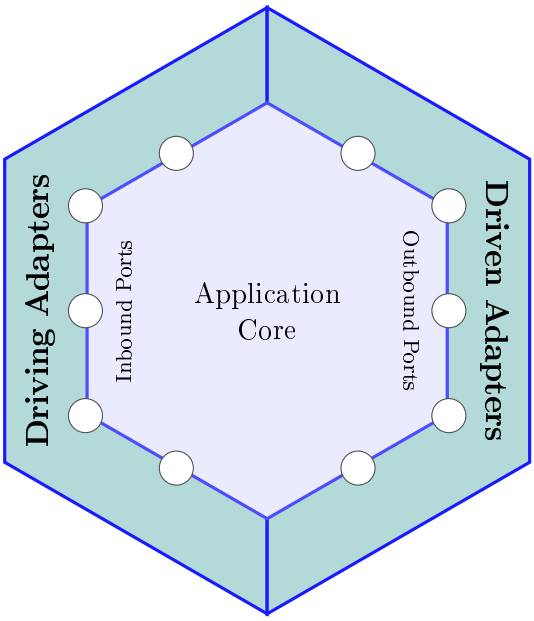


Figure 1.6: Driven adapters to the right of the application core.

And, again, we can update the diagram to include a thin hexagon around the core to represent the ports of the core. Just to clarify that there is a "border" between the core and the adapters, on both sides.

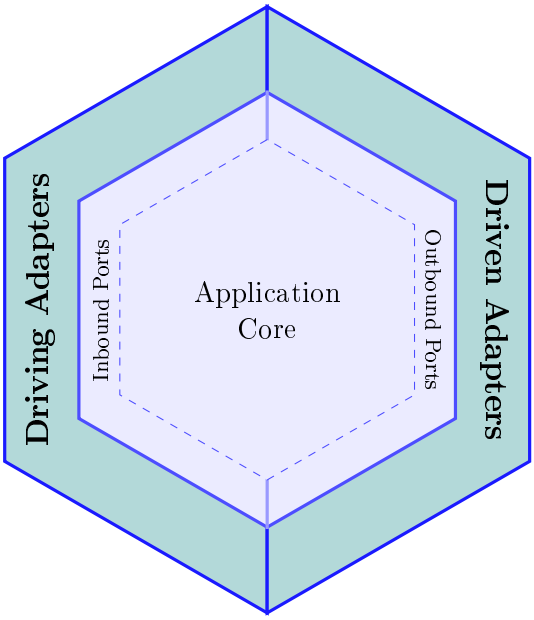


Figure 1.7: Both sides of the hexagon, with the core in the middle.

Notice how the inner hex is dashed, to indicate a "border" between the core and its ports, but to still indicate that the ports are *inside* the core.

1.4.1 Examples of Driven Adapters

Typical driven adapters are:

- a repository adapter for `PostgreSQL` or `MongoDB`,
- an email adapter for `SMTP`,
- a payment adapter for an external payment service API, like Stripe or PayPal,
- and a publisher adapter for a message broker, like Kafka or RabbitMQ.

1.4.2 Outbound Ports and a Driven Adapter in Java

On the driven side, the core already defines outbound ports, and driven adapters implement them.
Let's start with the first outbound port, `LoadBreachPort` (Code snippet 1.4). It is used to load a Breach from storage. I use `Optional` to indicate that the Breach may not be found.

IN-REF: add reference, when Optional pattern is written

```
1 public interface LoadBreachPort
2 {
3     Optional<Breach> loadById(BreachId breachId);
4 }
```

Code snippet 1.4: `LoadBreachPort` outbound port

Then the next port, `SaveKaijuPort` (Code snippet 1.5), to save a Kaiju entity to the storage (database or other storage). I keep the interfaces small here. But sometimes you may want to group multiple related operations into a single interface. For example, you could have an interface which contains both the save and load operations.

```java
public interface SaveKaijuPort
{
    void save(Kaiju kaiju);
}
```

Code snippet 1.5: `SaveKaijuPort` outbound port

And an implementation of the latter port in Code snippet 1.6. Here we use `Spring Data JPA` to save the Kaiju to storage. If you are unfamiliar with `Spring Data JPA`, you can think of it as a library that provides a way to interact with a database, without explicitly writing SQL queries.

```java
public final class JpaKaijuRepositoryAdapter
    implements SaveKaijuPort
{
    // Dependency injected.
    private final SpringDataKaijuRepository repository;

    @Override
    public void save(Kaiju kaiju)
    {
        repository.save(KaijuEntity.fromDomain(kaiju));
    }
}
```

Code snippet 1.6: `JpaKaijuRepositoryAdapter` driven adapter

The purpose of this adapter is to hide the database implementation details from the core.

1.4.3 Mapping Boundaries in Java

Mapping code belongs in adapters, because adapters translate between outside models and core models. Code snippet 1.7 shows request and command mapping; Code snippet 1.8 shows the persistence-side entity. For simplicity, I use here record classes to represent the commands and requests.

```
public record KaijuDetectionRequest(
    String codename,
    KaijuClassification classification,
    BreachId originBreachId
) {}

public record ConfirmKaijuDetectionCommand(
    String codename,
    KaijuClassification classification,
    BreachId originBreachId
) {}

ConfirmKaijuDetectionCommand command =
    new ConfirmKaijuDetectionCommand(
        request.codename(),
        request.classification(),
        request.originBreachId()
    );
```

Code snippet 1.7: Request and command mapping records

```
@Entity
public class KaijuEntity
{
    @Id
    private String id;
    private String codename;

    public static KaijuEntity fromDomain(Kaiju kaiju)
    {
        KaijuEntity entity = new KaijuEntity();
        entity.id = kaiju.id().value();
        entity.codename = kaiju.codename();
        return entity;
    }
}
```

Code snippet 1.8: `KaijuEntity` JPA persistence model

1.4.4 Dependency Rules

We must remember the dependency rules of this architecture

- The core should not depend on the adapters.
- The core defines the ports (interfaces) that the adapters must implement.
- The adapters should depend on the core.
- The adapters should not depend on each other.

This means that dependencies should only go from the outside towards to core. When we then

DIAGRAM: add diagram showing the direction of dependencies. Inner hex and outer hex divided vertically into two

organize the code into packages or modules, we apply these dependency rules by being careful about which packages a class can import.
With this layout, we keep dependency directions explicit:

- `domain` and `application` do not depend on `adapters`.
- `adapters.in.*` depend on inbound ports in `application.ports.in` (i.e. the interfaces providing access to the core). Classes in this package are our driving adapters.
- `application` depends on outbound ports in `application.ports.out` (i.e. the interfaces that the core defines, to be implemented by the adapters).
- `adapters.out.*` implement outbound ports and may depend on framework libraries. Classes in this package are our driven adapters.
- `config` performs wiring and can see both core and adapters.

Package naming makes these rules easier to spot in import statements, but teams can also automate them. For example, architecture tests in Java, such as ArchUnit, can fail the build if `application` or `domain` imports adapter packages, or if adapters depend on each other. As noted when discussing package layout, such tests are often straightforward once package roles align with the hexagon.

1.4.5 Requires vs Provides Contracts

The most important design choice on the driven side is contract ownership. This is where the hexagonal architecture differs from layered architecture. In hexagonal architecture, the core defines contracts by what the use cases *require*. For example, a core service may depend on `LoadBreachPort` and `SaveKaijuPort`, because those capabilities are needed to complete the use case and preserve the Kaiju-origin invariant.
In many layered implementations, interfaces are shaped by what the persistence layer *provides*, for example a repository abstraction aligned to ORM operations. That approach can still be effective, and we can still mock those interfaces. The difference is who drives the abstraction: in hexagonal architecture, required core behavior drives the port definition; in provider-oriented layering, available infrastructure capabilities often drive the interface definition.
This distinction matters during development. With core-owned outbound ports, we can implement and test use cases before deciding the final database, broker, or external API integration.

DIAGRAM: I don't like this diagram

1.5 Step 4: Evolving the Core with a Domain Model

In some projects, the first hexagonal version uses lightweight application services and simple data structures. That is a valid starting point. As rules become richer, we often add an explicit Domain Model to keep invariants and behavior close to business concepts.
Cockburn's original model did not include an explicit domain model, but if you were to study this architecture online, you will find that it is usually included.

1.5.1 Why the Domain Model May Come Later

In the original hexagonal architecture, there was no domain model. You *can* write software without a domain model of classes and objects. You might go with a simple *data model*, meaning a collection of classes that represent the data you are working with. I realize some people *do* consider this a domain model, but it is an *anemic domain model*, meaning only data but no behavior. You can also go even more primitive and just use simple data structures, but I really don't think anyone wants that.

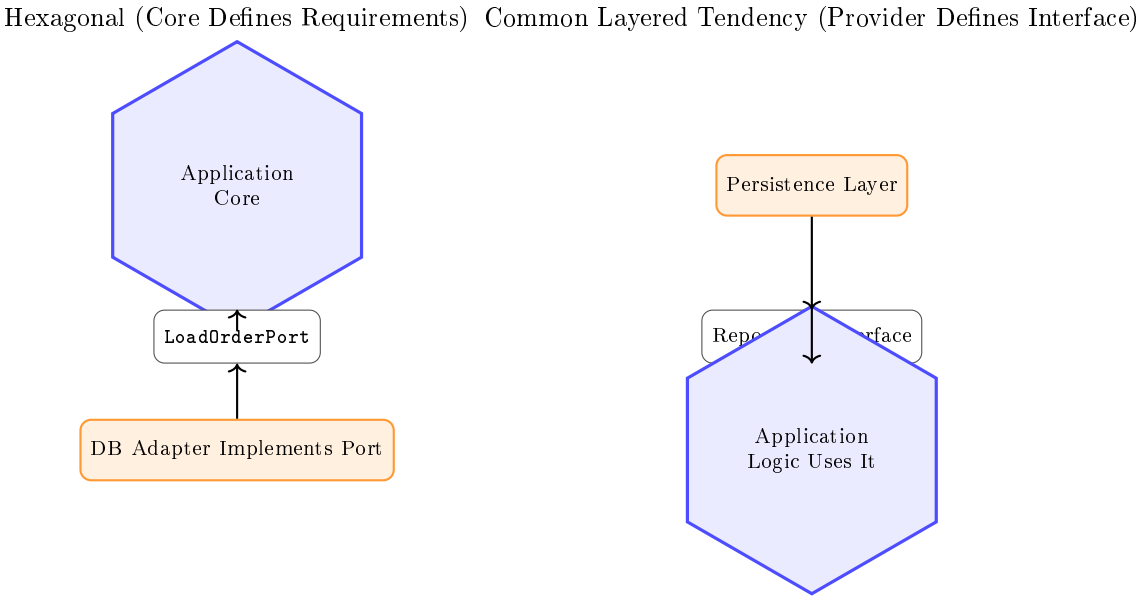


Figure 1.8: Contract ownership comparison: core-defined required ports vs provider-defined interfaces.

About the same time as hexagonal architecture was introduced, the Domain-Driven Design (DDD) paradigm was introduced by Eric Evans [2]. And so, shortly after, the original hexagonal architecture diagram was extended with a domain model.
Adding a domain model later is not a failure of architecture; it is often a sign of responsible evolution. We first establish boundaries and interaction patterns, then deepen the model when complexity justifies it.
We can illustrate the addition of the Domain Model by introducing a new hexagon, inside the core. This now means that the Service classes, in Application Core, can act upon the Domain Model, and that the Domain Model has no dependencies on any outer hexagon.
Figure 1.9 shows the updated diagram:
For the sake of clarity, I have added a nice arrow to indicate that the dependencies point inward. Outer hexes (layers) may depend on inner hexes, e.g. by importing and using classes or functions from the inner hex (package/module).
Another way to diagram the final architecture is the more common way of using boxes for each layer/part. This is shown in Figure 1.10:

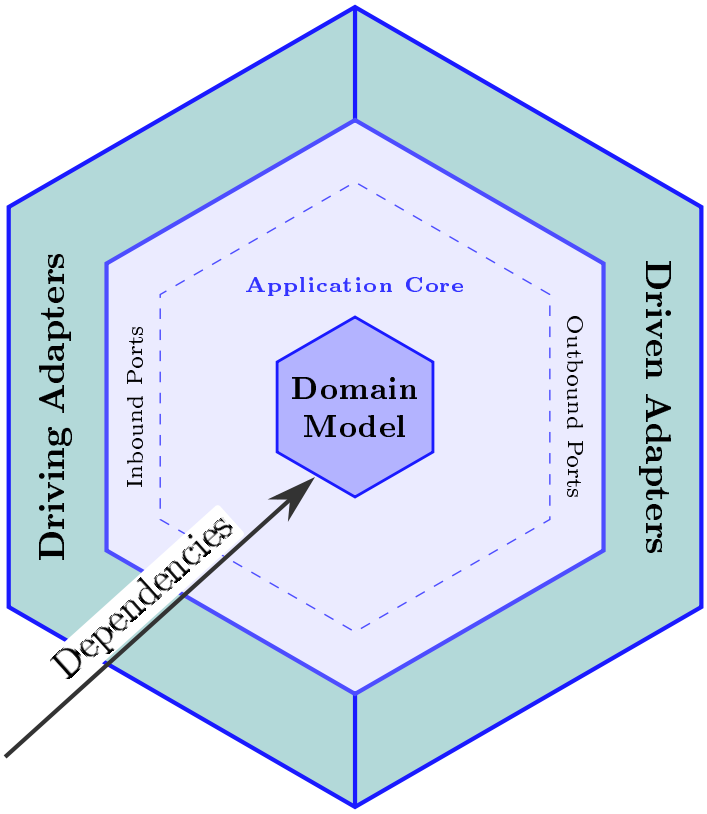


Figure 1.9: The domain model inside the core.

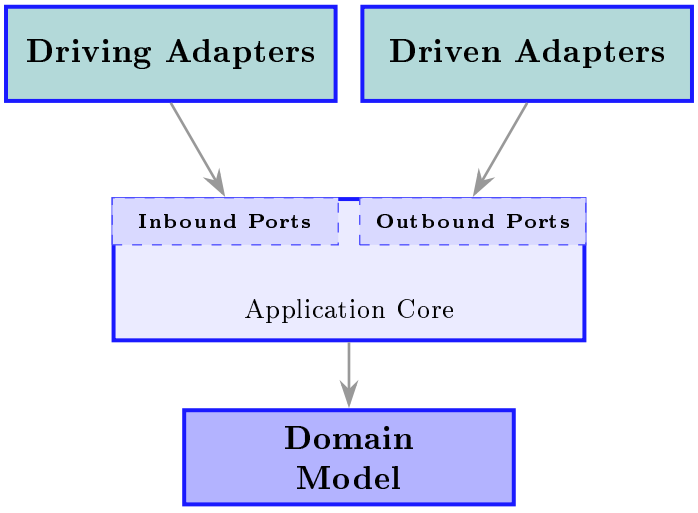


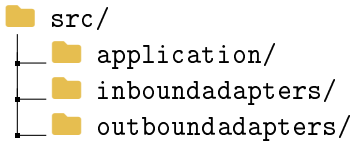
Figure 1.10: The final architecture as boxes.

1.6 Package Layout

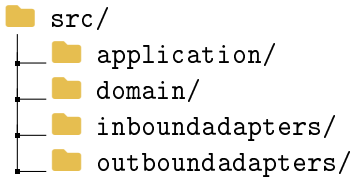
An important aspect of *architecture* is the organization of the code into packages and modules, along with the dependencies between them. Most architectural styles will have a recommended superficial package layout, so let us explore how we can organize an application using hexagonal architecture.
A package convention gives us a practical guardrail. We can often organize our code by architectural role, so that core code remains stable while adapters can change. There are many ways to do this, I will give an overall recommendation, and then two suggestions on how to organize the aplication hex: layers and feature-folders.

1.6.1 Overall structure

We generally have three parts to a hexagonal architecture: the core, the driving (inbound) adapters, and the driven (outbound) adapters. So, that's where we start: create three packages (or modules).



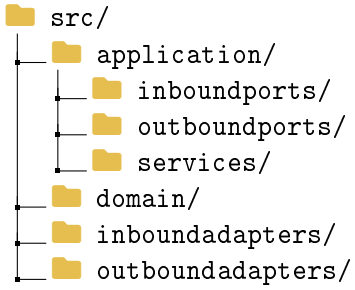
Many applications, across many architectural styles, will actually have a similar package structure, perhaps just differently named. Sometimes the application is called "core", sometimes "services". The same responsibility has been named different things over the years.
Next, we also need the domain. We can place it inside the application package, to represent that the domain hex is inside the application hex. Or, we can put it in a separate package. Below, I will show the latter:



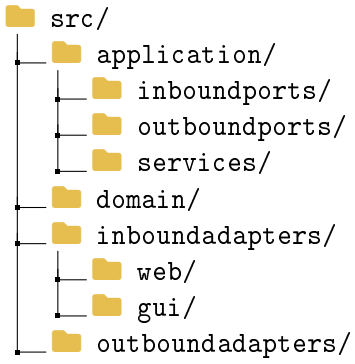
Then, we have to further organize the application hex, or package. As mentioned earlier, I will show two approaches: layers and feature-folders.

1.6.2 Application Layers

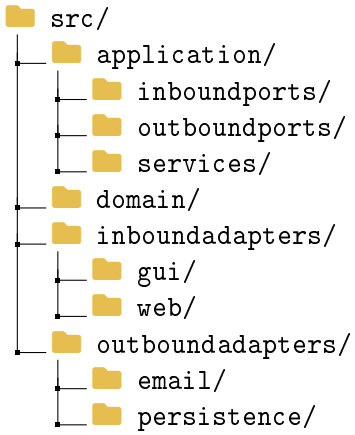
Let's start with layers. The application hex consists of service classes (the classes performing the use cases), inbound ports (the interfaces exposing the use cases to the outside world), and outbound ports (the interfaces defining the capabilities the core needs from the outside world). If we go with the layered approach, we group files by their type of responsibility, or category. That means three sub-packages:



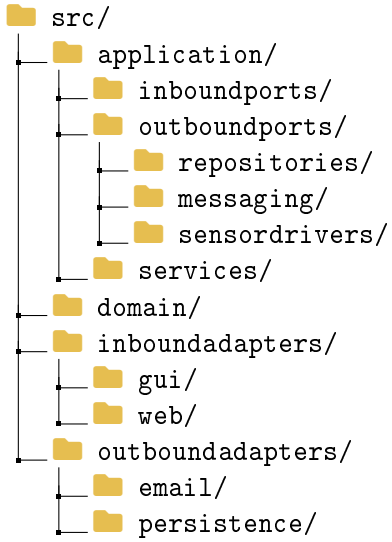
This is the basics. Your application may need other kinds of classes, which do not fit into one of the three categories.
I will not go into too precise details about the adapters here. In your `inboundadapters` package, you will have classes that lets the outside world interact with the application. This could be a Web API controller, a CLI handler, a GUI controller, a message consumer, or something else. Let's just say that our application exposes a REST API, and also has a GUI. I will make two sub-packages in the `inboundadapters` package: `web` and `gui`.



What about outbound adapters? Again, we have many categories, try to organize your code by your needs. For example, you may have packages for `persistence`, `events`, `notifications`, `messaging`, `email`, `logging`, `sensordrivers`, `externalAPIs`, etc. Let's just add a sub-package for `persistence` and `email`:

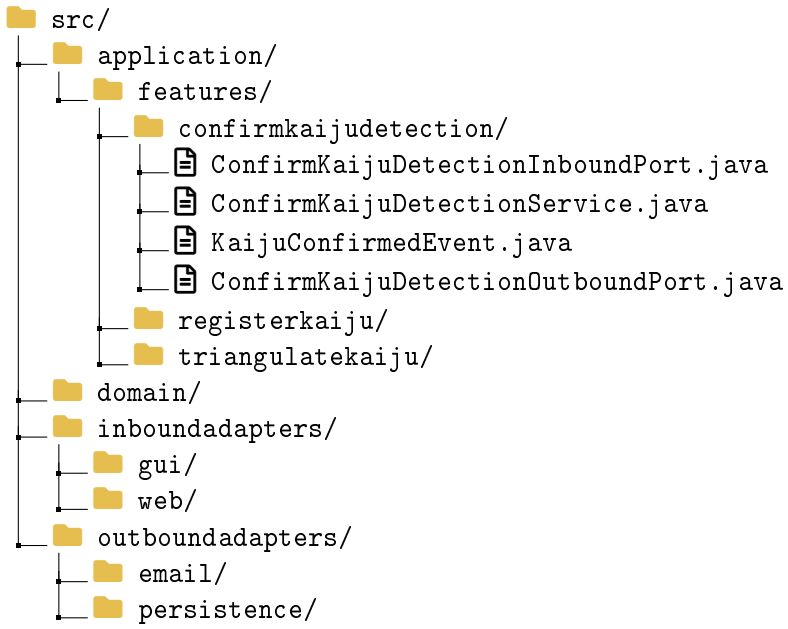


Further subdivision of your application is of course possible, and as the capabilities of the application grow, you will likely need to add more sub-packages. Here, you will have to decide what is the best way to organize your code.
For example, you may want a group of `repositories` ports, which are used to perform CRUD operations of entities on the database. You make a package for that: `outboundports/repositories`. Or you have ports for messaging, or sensor drivers. So, we can update the package structure to:



1.6.3 Feature Folders

Another way to organize the application is to use feature folders. In the previous approach, a service class in the `services` package will expose an inboundport, located in the `inboundports` package. And it may depend on a number of outbound ports, located in the `outboundports` package. That means, classes which work closely together to implement a feature, are spread across different packages, and navigating between those files may become cumbersome. That, at least, is the concern, which leads to the feature folder approach. So, instead, we organize the code by feature, or use case, a package per. And the service class, with its inbound and outbound ports, will be located in the feature package.
Here is an example, I have just updated the `application` package to use feature folders:



Notice how each feature package clearly describes the feature through its name. It then contains whatever classes and interfaces are needed to implement the feature. The adapters, of course, are still placed outside. Just pretend the other feature packages also contains relevant files.
You might similarly organize your adapter layers by feature, so in the `outboundadapters` package, you will have sub-packages `confirmkaijudetection`, `registerkaiju`, and `triangulatekaiju`.

And the same for the `inboundadapters` package.

1.6.4 Sub-layer folders vs Feature Folders

So, which approach should you choose? Both are valid, and both have their own advantages and disadvantages.

Layer folders: pros

- clear architectural map for newcomers, because `services`, `inboundports`, and `outboundports` make dependency direction explicit,
- easy to enforce package-level architecture rules and detect layering violations early, by simply looking at import statements,
- consistent structure across many features, which improves predictability in larger teams,
- centralized role-based code can simplify governance and cross-cutting standards,
- architecture tests are often straightforward, because package naming aligns with technical roles.
- reuse of outbound ports across services is easier, because multiple service classes in the service package can depend on the same outbound port.

Layer folders: cons

- feature code becomes scattered across several packages, increasing navigation overhead,
- implementing/updating one use case often requires changing files in multiple locations,
- classes that evolve together can drift apart because they are separated by technical category,
- extracting one capability later can be harder when its parts are distributed,
- you risk tying services together if they depend on the same outbound ports. One services requires a change to the port, now the other service may encounter issues.

Feature folders: pros

- high feature cohesion: service and ports for one use case live in one place,
- faster development flow, because most edits for a feature are local to one folder, or even just one file,
- ownership maps naturally to business capabilities rather than technical layers,
- clear organization by feature, making it easier to understand what the application does,
- updating a feature requires changes in *one* package,
- pull requests are often easier to review, because changes stay focused on one capability,
- feature-oriented boundaries can make future extraction or modularization easier, if you want to move towards a `modular monolith`, `vertical slice architecture`, or `microservices`.

IN-REF: add reference to modular monolith, vertical slice architecture, and microservices

Feature folders: cons

- repeated technical patterns can appear across features (validation, mapping, error handling) violating the DRY principle,
- conventions may drift if teams do not actively maintain shared standards, each developer may invent their own conventions,
- shared abstractions/ports need deliberate placement to avoid duplication or fragmentation, e.g. repository ports may be shared across features, and is often placed in a `common` package, which can grow larger and larger,
- global refactoring of cross-cutting concerns may touch many feature folders, which can be difficult to manage,
- you may consider organizing adapters similarly, which may make you wonder `why split a feature folder into three feature folders in the first place`? And that's a valid question. Or if you still organize your adapters by technical role, you are mixing two approaches. Will that be confusing?

A practical choice guideline

- prefer **layer folders** when architectural governance, role consistency, and shared technical abstractions are the main concern,
- prefer **feature folders** when business workflows change frequently and teams deliver vertical slices,
- and consider a **hybrid**: keep top-level feature folders, while applying light internal layering inside each feature where useful.

1.7 Example

I hope by now you have some kind of understanding of the architecture. Comparing it to the layered architecture, the main difference is primarily about where the outbound ports (interfaces) are placed. The rest is pretty similar to the layered architecture. So, I think it is time for another example. It will perhaps be a bit elaborate, because I had fun developing the use case. It is about using sensors to triangulate the position of a suspected Kaiju.
I will describe the use case, introduce the classes and ports involved, their relationships, and the package layout. I will also provide relevant code snippets. The focus will be on the structure of the code, primarily around the application hex, i.e. the service class. The inbound adapter could just be a Web API endpoint, which is not particularly interesting. And some of the outbound adapters would become too technical to be interesting, for example how to poll the sensors for data.
Furthermore, the specific purpose of the hexagonal architecture is to focus on the application logic, and treat the external adapters as technical details. You should notice that the shape of the outbound port is defined by what the application service *needs*. An adapter will then have to figure out how to actually achieve that, for example by connecting to a sensor and polling it for data. This is the port-first workflow from Section 1.2.1: we agreed on the port contract before anyone committed to a particular sensor protocol.

1.7.1 Use Case: Triangulate Kaiju Position

Here is the use case: Suppose we suspect a Kaiju has exited a Breach, and we want to triangulate its position. We have sensors placed across the sea, sensors of various types: Seismic, Acoustic,

Radiation, Thermal, etc. We want to select a relevant number of sensors in the suspected area, and use their readings to triangulate the position of the Kaiju. The result is an estimated area of the Kaiju, and we can then launch a Scout Vessel to investigate, and possibly visually confirm the Kaiju.

1.7.2 Visual explanation

As mentioned earlier, I picked this case because it is a bit elaborate, and I had fun developing it. It also (I hope) illustrates the concept of the hexagonal architecture well. But it does require me to introduce a bit of domain terminology first, so let's start with that.
Let's assume that any sensor, regardless of type, can return a reading which includes a direction (relative to the sensor's location), and a distance. This would result in a point on a 2d plane, i.e. the position of the Kaiju. However, sensors have some uncertainty in their readings, for example because of heavy waves, which may distort readings, or sea floor conditions like reefs, rocks, sand, etc, which may also distort readings. Therefore, we do not get a point, but an area. This area is called a *Annular Sector*.
And what exactly is an annular sector? Observe Figure 1.11.

A) Here we have a sensor, in the bottom left corner.

B) Here is the Kaiju, we suspect has appeared.

C) The sensor will produce a measurement, i.e. a vector, relative to the sensor's location.

D) The sensor has angular uncertainty, of some degree θ.

E) The direction is therefore a circular sector.

F) The sensor has distance uncertainty, of some radius `r`.

G) This results in two circular sectors (min and max distance).

H) In the end, we end up with the annular sector (red shaded area).

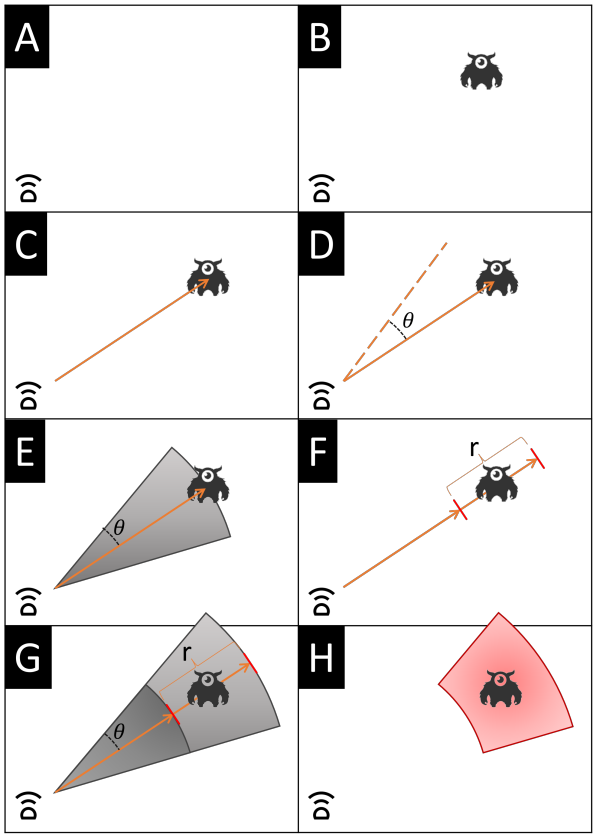


Figure 1.11: Deriving the annular sector.

Now, let's assume we have multiple sensors, each producing an annular sector. We can then use the intersection of these sectors to get a more accurate estimate of the Kaiju's position. This is the principle of triangulation. See Figure 1.12. Based on the sensor data from 5 different sensors, we have narrowed down the possible position of the Kaiju to a small area, marked in purple.

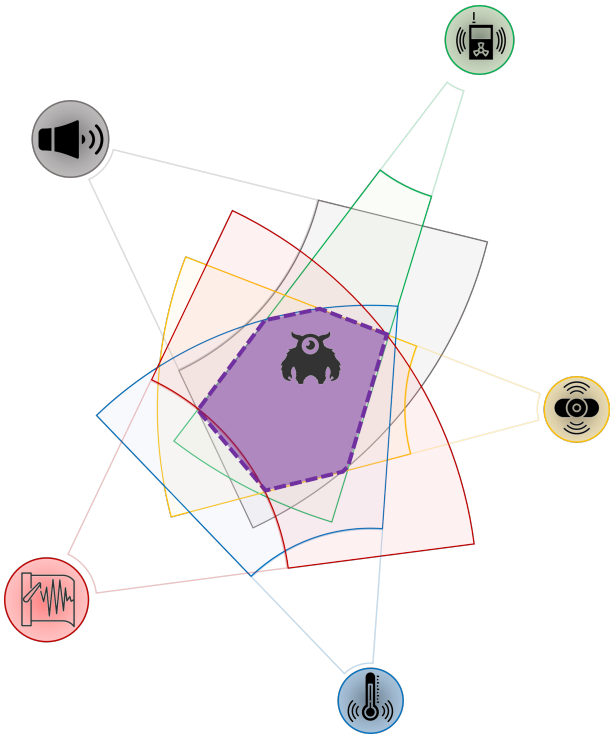


Figure 1.12: Triangulation of multiple annular sectors.

Good, I trust that the idea of the use case is now clear. The input to the use case is a suspected location: a point, plus a radius. All sensors within that radius will be polled. The output is an estimated area, wherein the Kaiju is likely to be. Another use case could then be to visually confirm the Kaiju, by launching a Scout Vessel to the estimated area. But, one thing at a time. Let's start with the triangulation use case.
I will try to balance the complexity by abstracting away some technical details, like how to calculate the output area from the array of annular sectors. Let's just assume, we can write a method that takes an array of annular sectors, and returns an estimated area, wherein the Kaiju is likely to be.

1.7.3 Classes involved

Let's look at the classes involved, I will provide a brief overview. Follow up sections will show the structure or location of the code, in relation to the hexagonal architecture. As this architecture focuses on business logic first, let's derive the classes for the use case by moving from inner hexes to outer hexes.

Domain

Let's start with the inner most hexagon, the domain.

1. I need a domain entity to represent the `Sensor`. It will contain various values, like `SensorId`, `SensorType`, and more, and some of these will be represented by other classes. You will see in the code later.
2. To avoid `primitive obsession`, I am going to model this with a few other classes, like `Location`, `Longitude`, `Latitude`, `SensorId`, `SensorType`, and perhaps more.

IN-REF: add reference to primitive obsession

Application

Then we move on to the application hexagon, where I define services and ports.

1. I need the `Service` class to implement the use case, `TriangulateKaijuPositionService`. It will contain the business logic, and will depend on the `Sensor` entity. It will have a method for executing the use case, which takes a `Location`, and a `Radius`. It will return a class, `EstimatedArea`, to represent the estimated area of the Kaiju. This `EstimatedArea` is not a domain entity, but just a simple object with a few properties.

2. The `Service` class will need to be able to retrieve all sensors within a given radius. I will create a `SensorRepository` port for this. Remember, a `Repository` is generally a port which can perform CRUD operations on an entity against a data storage (e.g. database), and the `Read` operation may optionally take filter parameters, like location and radius. Alternatively, I would define a dedicated port just for this use case.

3. The `Service` class will need to be able to poll the real sensors for data. The `Sensor` entity is just a *representation* of a real sensor somewhere out in the ocean. The domain is isolated from the outside world, and therefore nothing in the domain can access the outside world, or more specifically, my `Sensor` entity cannot access the real world sensor. Instead, I need a port to represent the ability to poll the real sensor for data. I will call this port `SensorDriver`. The interface is shaped by the use-case need to poll sensors and receive readings, not by any particular vendor protocol; there will be multiple adapters, which can connect to the real sensors.

4. Because I have different types of sensors, based on the `Sensor`'s `SensorType`, there will be a specific adapter of the `SensorDriver` port for each sensor type. I need a way to figure out which adapter to use. Here comes an interesting problem. I have multiple adapters to the same port, the `SensorDriver` port. How do I know which adapter to use? I am going to introduce a `SensorDriverFactory` port, which will be responsible for creating the correct adapter based on the `SensorType`.

5. I also need to expose the use case through a port, so I create an interface for the service class, let's call it `TriangulateKaijuPositionUseCase`.

Inbound Adapters

We need access to the system, so we can execute the use case. I will create a Web API endpoint for this, but it could also be a CLI action, or a GUI button, or a message consumer, or something else. This is a technical detail, and therefore placed outside the core. And also less interesting to write about. So, let's just say I have a Web API endpoint, which will be responsible for invoking the use case.

Outbound Adapters

These adapters provide access to the outside world. Either so `service` class can send data out of the system, or pull data into the system.

1. I need an adapter for the `SensorRepository` port. Let's assume several use cases need to execute CRUD operations on sensor entities, so there is already an adapter for this port.

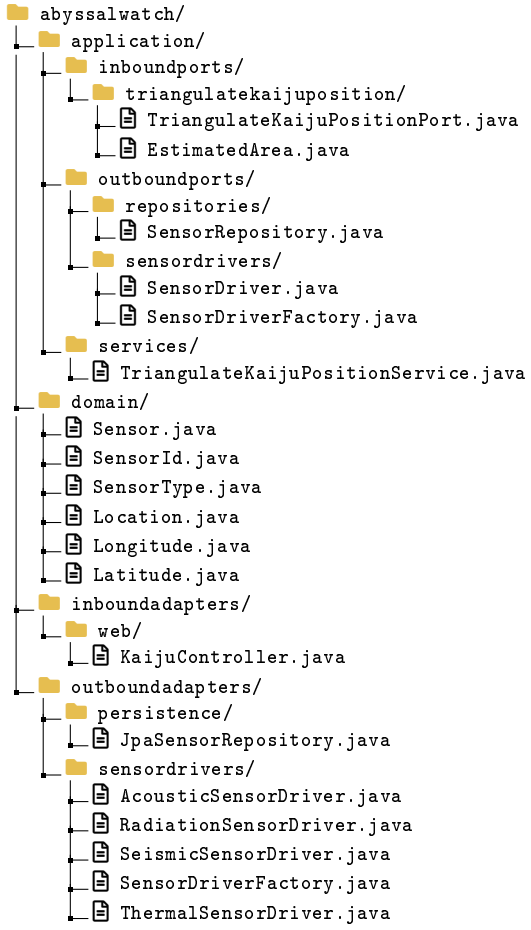
2. I need an adapter for the `SensorDriver` port. Actually, as I mentioned earlier, I need multiple adapters for this port, one for each sensor type. Different sensor types may require different protocols to communicate with the real sensor.

3. I need an adapter for the `SensorDriverFactory` port. It will have a method for creating the correct adapter based on an input of a `SensorType`. It then figures out which adapter to create, and return it. Remember, all my SensorDriverAdapters will implement the `SensorDriver` port, so they will all have the same interface.

I just need to make sure it is clear, why the `SensorDriverFactory` adapter is needed, and why it is located here in the outbound adapters. My `Sensor` entity just knows its type, but the specifics about how to connect to the real sensor is not part of the domain, we use the `SensorDriverAdapters` to do that. And these are located in the outbound adapters.
My Factory needs to know about the concrete adapters, and because the Application hex cannot know about the Outbound hex, the Factory *must* be placed in the Outbound hex, where it can reference, and thereby create, the concrete adapters. My Application hex still needs access to this adapter, so we define the port in the Application hex.
These are the classes involved, so let's now look at the package layout.

1.7.4 Package layout

For this example I use the layered organization of the application hex, that is, separate sub-packages for `inboundports`, `outboundports`, and `services`. The outbound ports are further grouped by category, so that repository ports and sensor driver ports each live in their own sub-package. Domain types and adapters follow the same overall structure introduced earlier in this chapter.
Notice how the `SensorDriverFactory` sits next to the concrete `SensorDriver` adapters in `outboundadapters/sensordrivers/`, while its port, `SensorDriverFactory`, stays in `application/ outboundports/sensordrivers/`.
The factory must be able to reference each concrete adapter to instantiate the right one, which is only allowed on the outbound side. The next subsection shows how these classes relate to each other.
You should also notice the location of the `EstimatedArea` class. Why is this placed in the inboundports package? Because it is a DTO, a simple data transfer object. It contains the result of the use case. It is therefore part of the *contract* for the use case. The inbound adapter hexagon should depend on this `inboundports` package. It should not depend on anything else in the application hexagon. Therefore, everything that is exposed to the outside of the application package, must be placed in the inboundports package.
If you have a class to act as the input for the use case, it should also be placed here.



1.7.5 Package diagram

One way to visualize the application structure is with a folder structure diagram, as we just saw. This shows grouping and organization of classes. But it does not show the relationships between the classes, and by extention, the relationships between packages. The relationships and dependencies between different parts of the application is very important to manage, when we talk architecture.
I will start with a simple package diagram, see Figure 1.13. The dashed arrows between packages indicate that a class in one package depends on a class/interface in another package. Essentially,

there is an `import` statement in the code. I use different shades for the packages based on their depth, just a personal preference. I believe it adds a bit of clarity. I organize the package layout stacked vertically, as this is general convention. Classes handling input to the system are placed at the top, and classes handling output from the system are placed at the bottom.
Observe, how the dependency arrows point:

- from the `application` package to the `domain` package,
- from the `inboundadapters` package to the `application` package,
- from the `outboundadapters` package to the `application` package.

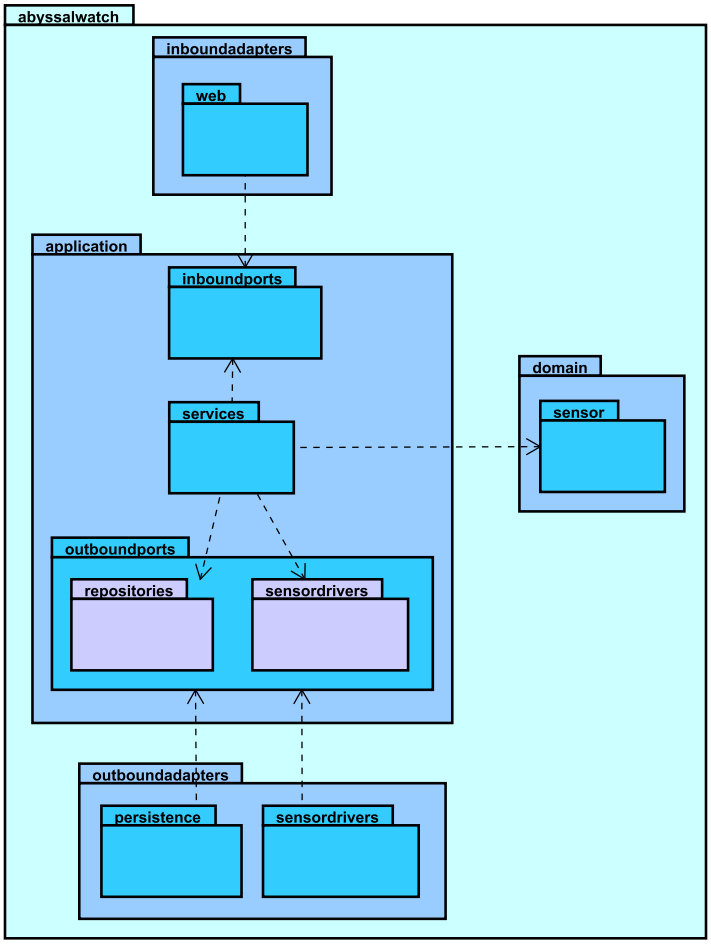


Figure 1.13: Package diagram of the application.

1.7.6 Class diagram

Next, we zoom in an extra step, so let's elaborate on the diagram, by adding the classes and their relationships. To keep things simple, I will leave out details like methods and fields. For now, we just focus on the classes, interfaces, and their relationships.

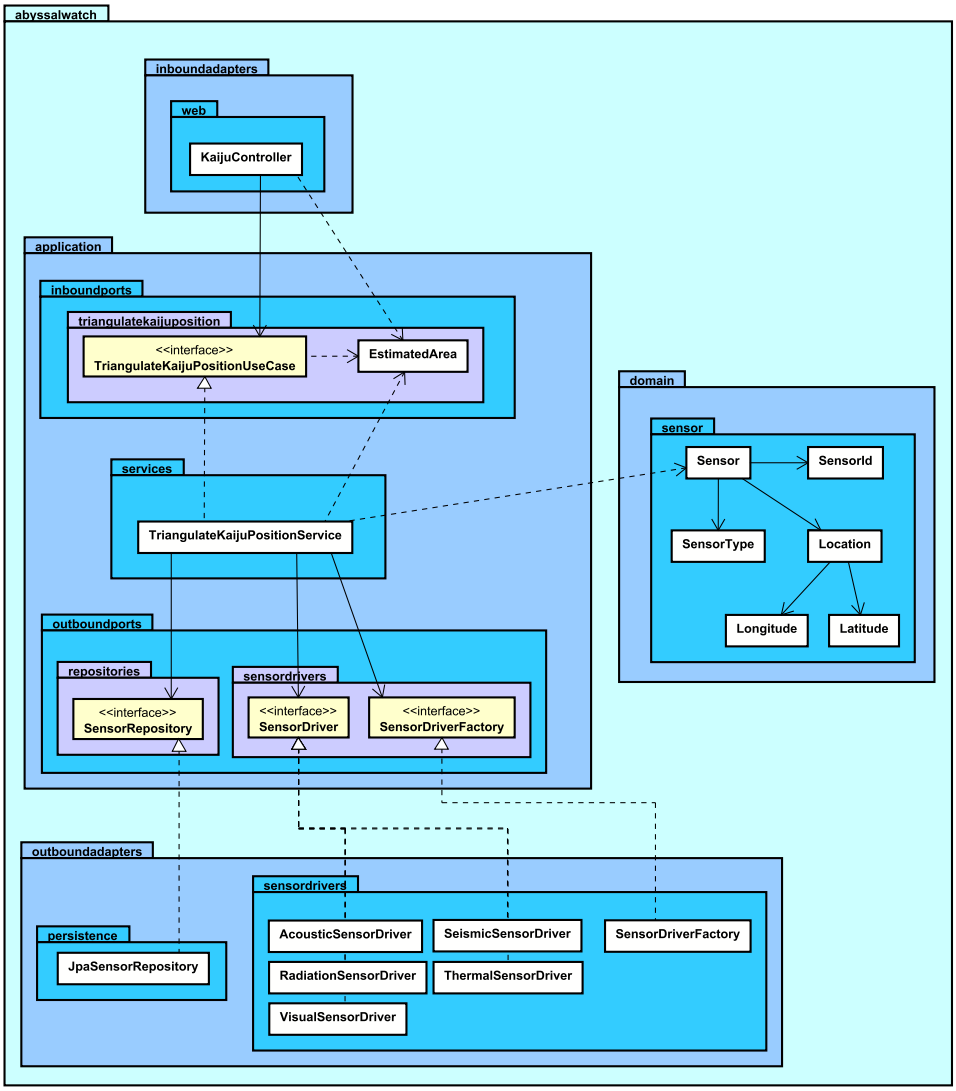


Figure 1.14: Package diagram of the application.

In Figure 1.14, the arrows are more explicit: different arrows for different types of connections. But still, between packages, they point the same direction as the previous package diagram in Figure 1.13.

- We have arrows *from* `outboundadapters` *to* `application`, now in form of *implements*. But that is still a kind of dependency.
- We have arrows *from* `inboundadapters` *to* `application`, now in form of *associations* and *dependencies*. Both, obviously, are kinds of dependencies.

It is time to move from design to implementation. Let's follow up with some code examples.

1.7.7 Implementation

The example have grown fairly comprehensive by now, but let's try and a look, again hex by hex. I reserve the right to abstract away some details here and there to shorten things.

Domain

First, I model the domain classes. To keep things short and simple, I will do this with records. The result is that the domain classes are very simple, just data, and contains no behavior or logic. I could do a rich domain-model, with behaviour, but that is not the point of this example, and I want to keep some work in the service class, as that closer follows the original presentation of the architecture.
You can of course keep the classes, but just move some logic from service to domain entity, if you prefer. For now, see Code snippet 1.9.

```
public record Sensor(
    SensorId id,
    SensorType type,
    Location location,
    double uncertaintyRadius,
    double uncertaintyAngle
);

public record SensorId(String value);

public record SensorType(String value);

public record Location(
    Latitude latitude,
    Longitude longitude
);

public record Latitude(double value);

public record Longitude(double value);
```

Code snippet 1.9: Sensor entity, and supporting values

The point of all these small records is to have explicit types for the values. The alternative was to have the `Sensor` class take 4 strings and 2 doubles, which would be less expressive.

Application

We move on to the application hexagon. As previously mentioned, this hexagon consists of three parts: inbound ports, outbound ports, and service classes. So, let's split this up.

Inbound ports: First, the inbound port and data in Code snippet 1.10. At the time of writing, I am unsure how to structure the data for the `EstimatedArea` class. It is some irregular polygon, but I will just represent it as a list of points, or locations.

```java
public interface TriangulateKaijuPositionPort
{
    EstimatedArea triangulate(Location location, double radius);
}

public record EstimatedArea(List<Location> points);
```

Code snippet 1.10: `TriangulateKaijuPositionPort` and `EstimatedArea`

Outbound ports: Let's then move on to the outbound ports in Code snippet 1.11. I need three interfaces:

```java
public interface SensorRepository
{
    List<Sensor> findAllByCircle(
        Location location,
        double radius
    );
}

public interface SensorDriver
{
    AnnularSector poll(Location targetLocation);
}

public interface SensorDriverFactory
{
    SensorDriver create(SensorType type);
}
```

Code snippet 1.11: Sensor repository and driver outbound ports

Service class: Now that the interfaces are in place, I can implement the use case in `TriangulateKaijuPositionService`. The class is long enough that I show it in five snippets. First, the class, of course, implements the inbound port declared above in Code snippet 1.10. Code snippet 1.12 shows how the constructor wires in the outbound ports from Code snippet 1.11.

```java
public class TriangulateKaijuPositionService implements
    TriangulateKaijuPositionPort
{
    private final SensorRepository sensorRepository;
    private final SensorDriverFactory sensorDriverFactory;

    public TriangulateKaijuPositionService(
        SensorRepository sensorRepository,
        SensorDriverFactory sensorDriverFactory
    )
    {
        this.sensorRepository = sensorRepository;
        this.sensorDriverFactory = sensorDriverFactory;
    }
    // methods follow in subsequent snippets
}
```

Code snippet 1.12: `TriangulateKaijuPositionService` dependencies and constructor

The `triangulate` method in Code snippet 1.10 orchestrates the use case: it loads sensors in the given radius, delegates to helper methods, and returns an `EstimatedArea`.

```java
@Override
public EstimatedArea triangulate(
    Location location,
    double radius
)
{
    List<Sensor> sensors =
        sensorRepository.findAllByCircle(
            location,
            radius
        );

    List<SensorDriver> sensorDrivers =
        createSensorDrivers(sensors);

    List<AnnularSector> annularSectors =
        pollSensorDrivers(sensorDrivers, location);

    List<Location> estimatedPoints =
        computeEstimatedArea(annularSectors);

    return new EstimatedArea(estimatedPoints);
}
```

Code snippet 1.13: `triangulate` use-case orchestration

The `SensorDriverFactory` creates the correct `SensorDriver` adapter for each `SensorType`, this is done in the helper method `createSensorDrivers` in Code snippet 1.14.

```
private List<SensorDriver> createSensorDrivers(
    List<Sensor> sensors
)
{
    List<SensorDriver> sensorDrivers = new ArrayList<>();

    for (Sensor sensor : sensors)
    {
        SensorDriver driver =
            sensorDriverFactory.create(sensor.type());
        sensorDrivers.add(driver);
    }

    return sensorDrivers;
}
```

Code snippet 1.14: `createSensorDrivers` helper

Each driver is polled at the target location, see the helper method `pollSensorDrivers` in Code snippet 1.15. The readings are collected as `AnnularSector` values. I imagine the driver automatically uses the uncertainty radius and angle of the sensor entity, to convert the reading to an `AnnularSector`.

```
private List<AnnularSector> pollSensorDrivers(
    List<SensorDriver> sensorDrivers,
    Location location
)
{
    List<AnnularSector> annularSectors = new ArrayList<>();

    for (SensorDriver driver : sensorDrivers)
    {
        AnnularSector sector = driver.poll(location);
        annularSectors.add(sector);
    }

    return annularSectors;
}
```

Code snippet 1.15: `pollSensorDrivers` helper

The final piece of the triangulation logic, converting several annular sectors into a polygon defined by a list of locations, is done in the helper method `computeEstimatedArea` in Code snippet 1.16. This particular piece of mathematical logic is out of scope for this chapter; the helper is left as a stub.

```
private List<Location> computeEstimatedArea(
    List<AnnularSector> annularSectors
)
{
    // TODO: Implement this.
}
```

Code snippet 1.16: `computeEstimatedArea` helper (stub)

Together, Code snippets 1.12–1.16 form the full service class.
This concludes the example of the Triangulate Kaiju Position use case.

1.8 Testing Across the Hexagon Boundary

The proposed separation of concerns in the hexagonal architecture gives us a practical testing strategy:[4]

- test application behavior through inbound ports *without* real infrastructure (i.e. replace driven adapters with test doubles),
- test adapters against their technologies in focused integration tests,
- and keep end-to-end tests for wiring and critical user paths.

In short, we can focus on unit testing the behaviour of the core, by mocking the driven adapters.

IN-REF: add reference to mocking

1.8.1 Why Ports Improve Testability

Ports (that is, interfaces) improve testability because they create explicit *seams* around behavior, which in turn allow us to isolate the behavior we want to test. An inbound port gives us a stable way to invoke a use case, while outbound ports let us replace infrastructure (outbound adapters) with in-memory test doubles .

IN-REF: add reference to seams

IN-REF: add reference to test doubles

In practice, we execute a use case through an inbound port and provide fake implementations for the outbound ports such as `LoadBreachPort` or `PublishKaijuConfirmedPort`. This keeps most tests fast and deterministic, because they verify business rules without requiring a running database, message broker, or HTTP communication.
Do **note** that this is not exclusive to hexagonal architecture. A disciplined layered architecture can also isolate dependencies behind interfaces and achieve good tests. In fact, any kind of architecture, assuming there a sufficient *seams* in place, is generally testable. Hexagonal architecture was just designed to make these seams a first-class design element around the core application, so isolated tests become the default way of working rather than something added later.

1.8.2 Testing Slices with Java

In Java, we can keep most behavior tests focused on the core. Code snippet 1.17 shows an example of testing a use case defined in a service class. We use a fake implementation of the repository and the event publisher. Both fakes implement the necessary interface.

IN-REF: I need to update the code, *ports are not right, repository, maybe

```java
@Test
void confirmsDetectionAndPublishesEvent()
{
    LoadBreachPort loadPort = new FakeLoadBreachPort();
    SaveKaijuPort savePort = new FakeSaveKaijuPort();
    PublishKaijuConfirmedPort publishPort =
        new FakePublishKaijuConfirmedPort();

    ConfirmKaijuDetectionService service =
        new ConfirmKaijuDetectionService(
            loadPort,
            savePort,
            publishPort
        );

    ConfirmKaijuDetectionCommand command =
        new ConfirmKaijuDetectionCommand(
            "Leatherback",
            KaijuClassification.CATEGORY_III,
            new BreachId("B-01")
        );

    KaijuId id = service.confirmDetection(command);

    assertTrue(savePort.savedKaijuIds().contains(id));

    assertTrue(publishPort
        .publishedKaijuIds()
        .contains(command.originBreachId().value()));
}
```

Code snippet 1.17: Unit test via inbound port with fake driven adapters

In the previous piece of code, I used specific *save* and *load* ports. Often, these are combined into a single port, i.e. a *repository*, or Data Access Object .
That choice is a matter of port granularity. Fine-grained, use-case-shaped ports such as `LoadBreachPort` and `SaveKaijuPort` express exactly what one use case needs. A shared `SensorRepository` makes sense when several use cases perform CRUD on the same entity. Alternatively, a dedicated port such as *find sensors within radius* avoids exposing full CRUD when the use case only needs one query. In all cases, shape the port by what the use case *requires*, as in Section 1.4.5; a repository is a reasonable choice when multiple use cases genuinely share the same persistence operations, not as a default CRUD port for every entity.

IN-REF: add reference to repository pattern

IN-REF: add reference to Data Access Object pattern

1.9 Practical Trade-offs

Hexagonal architecture is a deliberate investment in explicit boundaries, not a default choice for every system. It improves replaceability and testability, but introduces ports, adapters, and mapping code. For small CRUD-focused systems, that can feel heavy; for systems with changing workflows, complex logic, a rich domain, or expensive integration points, the added structure usually pays for itself over time.

1.9.1 Benefits

When the investment is justified, hexagonal architecture delivers several structural advantages:

- **Adapter replaceability:** persistence, messaging, and external APIs can be swapped without rewriting use cases, as motivated in Section 1.1,
- **Testability through explicit seams:** we can invoke inbound ports with fake driven adapters, as in Section 1.8.1 and Code snippet 1.17,
- **Core-owned outbound contracts:** ports express what use cases *require*, not what infrastructure *provides*, as discussed in Section 1.4.5,
- **Deferred infrastructure decisions:** we can implement and validate behaviour before committing to a database, broker, or sensor protocol, as in Section 1.2.1,
- **Multiple driving adapters:** a `REST` controller, `CLI` handler, `GUI` controller, or message consumer can share the same use case through inbound ports, as in Section 1.3,
- **Multiple driven adapter implementations:** several adapters can implement the same outbound port, and composition or configuration decides which one is used. In Section 1.7, multiple `SensorDriver` adapters satisfy one port for different sensor types; the same idea applies when notifications may go out by email, `SMS`, or a message broker through one port such as `PublishKaijuConfirmedPort`. For software that is sold to different customers, each deployment can wire its own adapter without changing core use cases,
- and **Enforceable dependency direction:** package and import rules can align with the architecture, as shown in Figure 1.13.

1.9.2 Costs

Those benefits come with real costs that teams should expect upfront:

- **Ceremony:** extra ports, adapters, commands (though, optional), DTOs, and mapping code compared with a thinner layered style,
- **More files to navigate:** one feature often spans core, ports, and adapters, and layer-folder layout can scatter code that changes together; see Section 1.6.4 for feature-folder and vertical-slice mitigations,
- **Slower early velocity** on simple CRUD where rules and integrations stay stable,
- **Learning and onboarding cost:** ports, adapters, and dependency direction must be understood and maintained,
- **Payoff depends on discipline:** without enforced boundaries, structure becomes ceremony, as noted in Section 1.9.4,
- and **Risk of over-structuring** small or short-lived systems that will not grow in complexity.

1.9.3 Things to Consider Before Deciding

Before adopting hexagonal architecture, it helps to match the style to the problem and the team. The lists below are not rigid rules, but practical signals.
Favour hexagonal architecture when:

- business workflows and rules change frequently,

- the domain has non-trivial invariants or coordination, such as linking a `Kaiju` to its origin `Breach`,
- there are multiple or volatile integration points, such as sensor drivers, payment APIs, or message brokers,
- you need several ways to trigger the same behaviour, for example a Web API, a batch job, and a `CLI`,
- isolated, fast tests of business rules matter,
- and the system is expected to live long enough for replaceability to matter.

Question or defer hexagonal architecture when:

- the system is mostly stable CRUD with few rules,
- there is one delivery mechanism and one database with little churn,
- the team cannot realistically enforce import and package rules,
- you are building a throwaway prototype or very small internal tool,
- and added interfaces would mostly duplicate pass-through calls with no real seams.

When uncertain, compare the signals in both lists and choose the lightest structure that still protects the behaviour you care about. You can also start hexagonal in a minimal form, perhaps one use case with a few ports, and deepen the model as complexity grows, without treating the style as something to migrate into from elsewhere.

1.9.4 Common Implementation Mistakes

Even with a good structure, implementations can drift. Ports and adapter folders are not enough on their own; the mistakes below usually show up as broken dependency direction, behaviour in the wrong layer, or structure that looks hexagonal but behaves like a layered architecture.
The first group of mistakes leaks infrastructure concerns into the core:

- placing framework annotations in core domain models and use cases,
- exposing `JPA` entities directly across the core boundary,
- defining repository interfaces around provider capabilities rather than use-case needs, as discussed in Section 1.4.5,
- embedding `HTTP` or `SQL` concerns in application services,
- letting the application core import concrete adapter classes instead of depending only on ports, which inverts the dependency rule shown in Figure 1.13,
- and surfacing infrastructure exceptions or types, for example `SQLException` or `HTTP` status codes, from use cases instead of domain- or application-level errors.

The next group breaks the intended flow between driving adapters, use cases, and driven adapters:

- driving adapters calling outbound ports directly and bypassing the use case, which recreates a thin-controller layered application with extra folders,
- putting business rules in controllers or persistence adapters while application services only delegate, i.e. fat adapters and an anemic core,

- passing transport or persistence DTOs into use cases instead of mapping to commands and results at the adapter edge, as in Code snippet 1.3,
- returning domain entities or persistence models from inbound ports when the outward contract should be explicit DTOs, as with `EstimatedArea` in Code snippet 1.10,
- and treating a REST controller, or similar driving adapter, as the inbound port rather than as one adapter behind a separate port interface.

Finally, some mistakes come from structure without discipline:

- adopting port and adapter folder names while import direction still violates the architecture,
- sharing one outbound port across unrelated features so a change for one use case breaks others,
- testing only through `HTTP` or a running database instead of invoking inbound ports with fake driven adapters, as in Section 1.8.1 and Code snippet 1.17,
- and applying full hexagonal structure to simple CRUD with little changing behavior, despite the trade-offs noted in Section 1.9.

Most of these mistakes are variations on the same theme: the core should express what a use case *requires*, and adapters should translate and implement those requirements. When that rule holds, the extra interfaces and mapping code from earlier in this chapter pay off; when it does not, the architecture adds ceremony without improving testability or replaceability.

1.9.5 Closing Thoughts

Hexagonal architecture organizes an application around one simple split: behaviour on the inside, technology on the outside. In this chapter we built that split step by step, starting with use cases in Section 1.2, adding driving adapters and inbound ports in Section 1.3, then driven adapters and outbound ports in Section 1.4, and optionally a domain model in Section 1.5.
We then looked at package layout in Section 1.6, and worked through two examples: `ConfirmKaijuDetectionService` for the basic shape, and the triangulation use case in Section 1.7 for richer integration complexity. Throughout, the governing idea from Section 1.4.5 stayed the same: ports are contracts the core owns, and adapters translate at the edges.
That structure is a deliberate choice, not a default for every system, as discussed in Section 1.9.3. It pays off when dependency direction is enforced in code and imports, as in Figure 1.13, and when we test behaviour through inbound ports, as in Section 1.8.1.
Hexagonal and layered architecture share the goal of separating behaviour from technology; hexagonal makes that separation explicit through ports and adapters, while layered architecture organizes code by technical role.
Feature folders or vertical slices, as in Section 1.6.4, can improve navigation without abandoning ports. In the end, the extra interfaces and mapping code earn their cost when boundaries are enforced, not merely named.

Bibliography

[1] Alistair Cockburn. *Hexagonal Architecture*. 2005. URL: https://alistair.cockburn.us/hexagonal-architecture (visited on 04/15/2026).

[2] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2003. ISBN: 978-0321125217.

[3] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. ISBN: 978-0321127426.

[4] Arho Huttunen. *Hexagonal Architecture Explained*. 2026. URL: https://www.arhohuttunen.com/hexagonal-architecture/ (visited on 04/15/2026).